

1. Foundations

Design and Analysis of Algorithms Important Textbook Parts

Below are few screenshots from the prescribed textbook that you all must read. Though they are covered in class through examples, below is the theory behind them.

1.1 What Is an Algorithm?

Although there is no universally agreed-on wording to describe this notion, there is general agreement about what the concept means:

An *algorithm* is a sequence of unambiguous instructions for solving a problem, i.e., for obtaining a required output for any legitimate input in a finite amount of time.

This definition can be illustrated by a simple diagram (Figure 1.1).

The reference to “instructions” in the definition implies that there is something or someone capable of understanding and following the instructions given. We call this a “computer,” keeping in mind that before the electronic computer was invented, the word “computer” meant a human being involved in performing numeric calculations. Nowadays, of course, “computers” are those ubiquitous electronic devices that have become indispensable in almost everything we do. Note, however, that although the majority of algorithms are indeed intended for eventual computer implementation, the notion of algorithm does not depend on such an assumption.

As examples illustrating the notion of the algorithm, we consider in this section three methods for solving the same problem: computing the greatest common divisor of two integers. These examples will help us to illustrate several important points:

- The nonambiguity requirement for each step of an algorithm cannot be compromised.
- The range of inputs for which an algorithm works has to be specified carefully.
- The same algorithm can be represented in several different ways.
- There may exist several algorithms for solving the same problem.

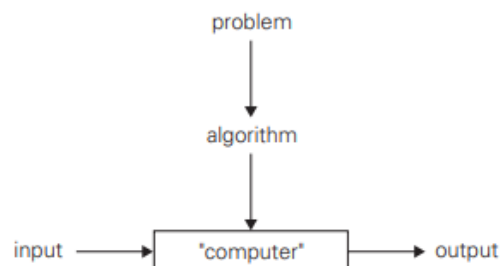


FIGURE 1.1 The notion of the algorithm.

Algorithm Design and Analysis Process:

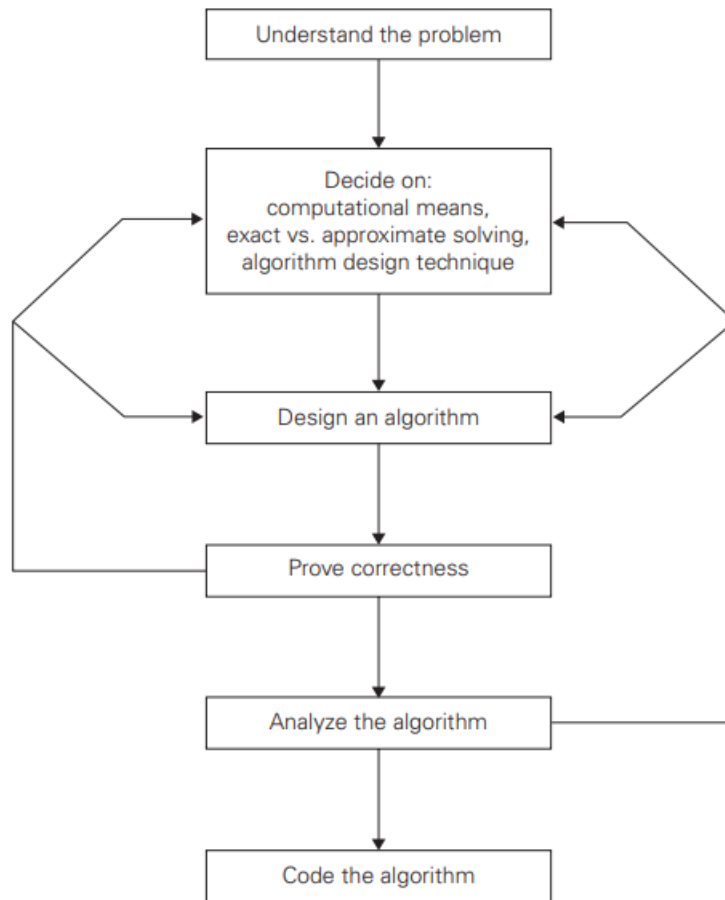


FIGURE 1.2 Algorithm design and analysis process.

Algorithm Properties:

An algorithm is a well-defined sequence of steps designed to solve a specific problem. To qualify as an algorithm, it must satisfy certain properties. Below are the key properties of an algorithm:

Input: An algorithm must accept zero or more inputs to work on.

Output: An algorithm must produce at least one output related to the input.

Definiteness: Each step in the algorithm must be clear, precise, and unambiguous.

Finiteness: An algorithm must terminate after a finite number of steps.

Effectiveness: Every step must be simple and feasible to execute using basic operations.

Generality: An algorithm must solve all problems of a specific type, not just one instance.

Correctness: An algorithm must produce the correct output for all valid inputs.

Efficiency: An algorithm must use minimal time and space resources to solve the problem.

Scalability: An algorithm should handle increasing input sizes gracefully.

Determinism: A deterministic algorithm always produces the same output for a given input.

General Plan for Analyzing the time of Non-Recursive and Recursive Algorithms

General Plan for Analyzing the Time Efficiency of Nonrecursive Algorithms

1. Decide on a parameter (or parameters) indicating an input's size.
2. Identify the algorithm's basic operation. (As a rule, it is located in the innermost loop.)
3. Check whether the number of times the basic operation is executed depends only on the size of an input. If it also depends on some additional property, the worst-case, average-case, and, if necessary, best-case efficiencies have to be investigated separately.
4. Set up a sum expressing the number of times the algorithm's basic operation is executed.⁴
5. Using standard formulas and rules of sum manipulation, either find a closed-form formula for the count or, at the very least, establish its order of growth.

General Plan for Analyzing the Time Efficiency of Recursive Algorithms

1. Decide on a parameter (or parameters) indicating an input's size.
2. Identify the algorithm's basic operation.
3. Check whether the number of times the basic operation is executed can vary on different inputs of the same size; if it can, the worst-case, average-case, and best-case efficiencies must be investigated separately.
4. Set up a recurrence relation, with an appropriate initial condition, for the number of times the basic operation is executed.
5. Solve the recurrence or, at least, ascertain the order of growth of its solution.